

สรุปโค้ดโปรเจกต์ TennisProject

และการแก้ไขปัญหาการตรวจจับลูกเทนนิส

เอกสารนี้อธิบายการทำงานของไฟล์ OpenVCPipeline.py และ Mapping.py
พร้อมสรุปปัญหาที่พบและขั้นตอนการแก้ไขตั้งแต่ต้นจนถึงปัจจุบัน

จัดทำวันที่ 24 สิงหาคม 2569 (2026)

1. ภาพรวมโปรเจกต์

โปรเจกต์นี้เป็นระบบวิเคราะห์วิดีโอกีฬาเทนนิสด้วยคอมพิวเตอร์วิทัศน์ (Computer Vision)

ประกอบด้วยสองขั้นตอนหลักที่ทำงานต่อเนื่องกัน:

- OpenVCPipeline.py - อ่านวิดีโอดิบ (เช่น video1.mp4) แล้วตรวจจับตำแหน่งเส้นสนาม ผู้เล่นทั้งสองฝั่ง และลูกเทนนิสในแต่ละเฟรม บันทึกผลเป็นวิดีโอกำกับกรอบ (detected_video_N.mp4) และไฟล์ CSV สองไฟล์ (court_coordinates_N.csv และ player_tracking_output_N.csv)
- Mapping.py - นำไฟล์ CSV จากขั้นตอนแรกมาแปลงพิกัดจากพิกเซลในภาพให้เป็นพิกัดจริงบนสนาม (หน่วยเมตร) ด้วยเทคนิค Homography คำนวณความเร็วลูกและระยะห่างผู้เล่น-ลูก แล้วแสดงผลเป็นแอนิเมชันแผนที่สนาม แบบ 2 มิติ (bird's-eye view) พร้อมกราฟประกอบ

2. อธิบายโค้ด OpenVCPipeline.py

2.1 การตั้งค่าโมเดล

บรรทัด 1-13

- ใช้โมเดล YOLOv8 (yolov8n.pt) ที่เทรนไว้ล่วงหน้าบนชุดข้อมูล COCO สำหรับตรวจจับ "คน" (class 0) และ "ลูกกีฬา / sports ball" (class 32)
- ใช้โมเดล YOLOv8-Pose (yolov8n-pose.pt) เพื่อหาตำแหน่งข้อเท้า (ankle keypoints) ของผู้เล่นฝั่งไกลกล้อง โดยเฉพาะ เนื่องจากผู้เล่นฝั่งไกลมีขนาดเล็กในภาพ การประมาณตำแหน่งเท้าจากขอบล่างกรอบธรรมดาจึงไม่แม่นยำ
- เคยทดลองเทรนโมเดลตรวจจับลูกเทนนิสของตัวเอง (custom model) มาก่อน แต่ผลไม่ดีพอ จึงใช้โมเดล COCO ทั่วไป (class 32) ต่อไป และแก้ปัญหาด้วยวิธีตรวจจับที่ปรับปรุงแล้วแทน (รายละเอียดในหัวข้อ 4-5)

2.2 ขั้นตอนที่ 0: ตรวจจับเส้นสนาม

บรรทัด 21-320 (โดยประมาณ)

กล้องอยู่ในระดับพื้นสนาม ไม่ใช่มุมสูงแบบ bird's-eye ทำให้เส้นข้างสนาม (sidelines) ในภาพถูบรบกวน ขณะที่เส้นหลังและเส้นเสิร์ฟยังดูเกือบเป็นแนวนอน แนวคิดหลักคือตรวจจับเฉพาะเส้นที่อยู่ใกล้กล้องและชัดเจน (เส้นหลังฝั่งใกล้ เส้นเสิร์ฟฝั่งใกล้ เส้นข้างสนามทั้งสองเส้น) แล้วใช้เทคนิคเรขาคณิตภาพเดียว (single-view metrology) แบบการประมาณค่านอกช่วงด้วยอัตราส่วนไขว้ (cross-ratio extrapolation) คำนวณหาตำแหน่งเส้นหลังและเส้นเสิร์ฟฝั่งไกล โดยไม่ต้องมองเห็นเส้นเหล่านั้นชัดเจนในภาพเลย

- `_find_court_blob` - หาบริเวณสีพื้นสนามด้วยฮิสโตแกรมสี (hue histogram) ในปริภูมิสี HSV เพื่อรองรับสนามที่มีสีต่างกัน
- `_lines_only_mask` - กรองเฉพาะพิกเซลสีขาว (เส้นสนาม) ภายในบริเวณสนามที่พบ
- `detect_court_points` - ใช้ Canny edge detection และ Hough Line Transform หาเส้นตรงทั้งหมด แบ่งเป็นเส้นแนวนอนกับแนวทแยง จัดกลุ่ม (clustering) เพื่อหาเส้นหลัง เส้นเสิร์ฟ และเส้นข้างสนามที่แท้จริง หาจุดตัดเพื่อได้มุมสนามฝั่งใกล้ (BL, BR, NSL, NSR) แล้วคำนวณจุดฝั่งไกล (TL, TR, FSL, FSR) ด้วยการประมาณค่าอัตราส่วนไขว้
- ผลลัพธ์บันทึกเป็นภาพตรวจสอบ (court_detection_debug.png) และบันทึกพิกัดเป็น court_coordinates_N.csv

2.3 การตรวจจับผู้เล่นฝั่งไกล (detect_far_player)

ผู้เล่นฝั่งไกลกล้องมีขนาดเล็กมากในภาพ (ประมาณ 50x30 พิกเซล) การตรวจจับตรง ๆ บนภาพเต็มมักได้ความมั่นใจต่ำ และบางครั้งสับสนกับวัตถุอื่นที่มีขนาดใกล้เคียง เช่น ผู้ชมหรือม้านั่งข้างสนาม

- วิธีแก้: ตัดเฉพาะบริเวณครึ่งสนามฝั่งไกล (far_court_crop_box) แล้วขยายภาพ 3 เท่า ก่อนส่งเข้าโมเดลตรวจจับคน ทำให้ความมั่นใจสูงขึ้นมาก และตัดปัญหาม้านั่ง/ผู้ชมออกไปโดยอัตโนมัติเพราะอยู่นอกกรอบที่ตัดมา
- ทดลองใช้ pose model เป็นตัวหลักก่อน แต่พบว่าพลาดการตรวจจับ 4 ใน 10 เฟรมทดสอบ ขณะที่โมเดลตรวจจับทั่วไปพบ ผู้เล่นครบทุกเฟรม จึงใช้โมเดลตรวจจับทั่วไปเป็นหลัก และใช้ pose model เสริมเฉพาะเมื่อพบข้อเท้าด้วยความมั่นใจสูงพอ

- กรองผู้เล่นที่ตรวจพบตามความสูงของกรอบ (ต่ำกว่า 100 พิกเซล)
เพื่อตัดกรณีผู้เล่นฝั่งใกล้เคียงเข้ามาในกรอบตัดภาพ จนถูกเข้าใจผิดว่าเป็นผู้เล่นฝั่งไกล

แนวคิดครอบภาพ + ขยาย (crop + upscale) นี้เป็นต้นแบบที่นำไปประยุกต์ใช้กับการตรวจจับลูกเทนนิสในภายหลังด้วย (หัวข้อ 5.5) เพียงแต่ผู้เล่นฝั่งไกลมีบริเวณที่แน่นอนตายตัว (ครึ่งสนามฝั่งไกล) ในขณะที่ลูกเทนนิสเคลื่อนที่ได้ทั่วทั้งสนาม จึงต้องครอบหลายจุดแทนที่จะเป็นจุดเดียว

2.4 การตรวจจับลูกเทนนิสแบบแบ่งส่วนภาพ (detect_ball_tiled)

ฟังก์ชันนี้คือทางออกสุดท้ายของปัญหาการตรวจจับลูกเทนนิสที่อธิบายละเอียดในหัวข้อ 5.5 หลักการคือแบ่งบริเวณที่เป็นไปได้ของลูก (plausible region) ออกเป็นชิ้นย่อย (tile) ขนาด 640x640 พิกเซล ซ้อนทับกันเล็กน้อย (overlap 100 พิกเซล) แล้วตรวจจับทีละชิ้นที่ความละเอียดต้นฉบับ (ไม่ย่อภาพเลย) จากนั้นรวมผลลัพธ์ทั้งหมดกลับเป็นพิกัดของภาพเต็ม

2.5 ลูปลักษณ์ของการประมวลผลวิดีโอ

อ่านเฟรมแรกเพื่อตรวจจับเส้นสนาม แล้วกำหนดขอบเขตพิกเซลที่ลูกเทนนิสควรอยู่ (BALL_X_MIN/MAX, BALL_Y_MIN/MAX) จากตำแหน่งมุมสนามที่ตรวจพบ จากนั้นในแต่ละเฟรม:

- ตรวจจับและติดตาม (track) ผู้เล่นด้วย YOLO class 0
- ตรวจจับลูกเทนนิสด้วย detect_ball_tiled (หัวข้อ 2.4) แทนการตรวจจับภาพเต็มเพียงครั้งเดียว
- กรองตำแหน่งลูกที่ตรวจพบให้อยู่ในขอบเขตที่สมเหตุสมผลของสนามเท่านั้น (plausible-region filter)
- ผ่านตัวกรองวัตถุนิ่ง (static-fixture filter) เพื่อตัดวัตถุที่ถูกตรวจพบซ้ำตำแหน่งเดิมบ่อยเกินไป (อธิบายละเอียดในหัวข้อ 5.1) เป็นเกราะป้องกันชั้นที่สอง
- เรียก detect_far_player สำหรับผู้เล่นฝั่งไกล
- วาดกรอบ/จุดกำกับทั้งหมดลงในเฟรม บันทึกเป็นวิดีโอผลลัพธ์ และเก็บพิกัดของเฟรมนั้นไว้บันทึกเป็น player_tracking_output_N.csv เมื่อจบวิดีโอ

3. อธิบายโค้ด Mapping.py

3.1 การโหลดข้อมูล

โหลดไฟล์ court_coordinates_N.csv (ตำแหน่งมุมสนามที่ตรวจพบ) และ player_tracking_output_N.csv (ตำแหน่งผู้เล่น/ลูกชายเฟรม) รวมถึงวิดีโอผลลัพธ์ (detected_video_N.mp4) เพื่อแสดงเทียบกับแผนที่ 2 มิติ และอ่านค่า fps จริงจากไฟล์ CSV แทนการสมมติค่าคงที่ เพื่อให้แอนิเมชันและการคำนวณความเร็วตรงกับวิดีโอต้นฉบับ

3.2 Homography: แปลงพิกเซลเป็นเมตร

- กำหนดค่ามาตรฐานสนามเทนนิสตามกติกา ITF (กว้าง 8.23 เมตร ยาว 23.77 เมตร ระยะเส้นเสิร์ฟ ฯลฯ)
- นำจุดมุมสนามที่ตรวจพบในพิกเซลมาจับคู่กับตำแหน่งจริงบนสนามเป็นเมตร แล้วใช้ cv2.findHomography คำนวณเมทริกซ์แปลงพิกัด (M) โดยใช้จุดที่ตรวจพบทั้งหมด (มุมสนาม + เส้นเสิร์ฟ) ไม่ใช่แค่ 4 มุม เพื่อลดผลกระทบจากความคลาดเคลื่อนของจุดใดจุดหนึ่ง

3.3 การกรองค่าผิดปกติ

- reject_pixel_track_outliers - ใช้สถิติ median + MAD (Median Absolute Deviation) ชิงหนานต่อค่าผิดปกติ มากกว่าค่าเฉลี่ย/ส่วนเบี่ยงเบนมาตรฐานทั่วไป เพื่อตรวจจับช่วงที่ผู้เล่นถูกติดตามผิดพลาด แล้วลบตำแหน่งเหล่านั้นออก จากนั้นใช้ interpolate() เติมค่าที่ขาดหายไป
- transform_to_meters - แปลงตำแหน่งจากพิกเซลเป็นเมตรด้วยเมทริกซ์ homography แล้วตรวจสอบอีกชั้นว่าตำแหน่ง ที่แปลงได้อยู่ในระยะที่สมเหตุสมผล (ไม่เกินขอบสนาม 5 เมตร) เนื่องจากบริเวณใกล้สูญหายไปในภาพ (vanishing point) การแปลงพิกัดจะอ่อนไหวต่อความคลาดเคลื่อนเล็กน้อยมาก

3.4 การปรับความเรียบของข้อมูล (Smoothing)

ใช้ rolling mean ขนาดหน้าต่าง 45 เฟรม (ประมาณ 0.75 วินาที) เฉพาะกับตำแหน่งผู้เล่นเท่านั้น ไม่ใช้กับลูกเทนนิส เพราะลูกเคลื่อนที่เร็วและเปลี่ยนทิศทางที่เมื่อโดนตี การเฉลี่ยจะทำให้ข้อมูลจริงเพี้ยนไป

3.5 ความเร็วลูกและระยะห่างผู้เล่น-ลูก

คำนวณระยะทางที่ลูกเคลื่อนที่ในแต่ละเฟรม (จากผลต่างตำแหน่งเป็นเมตร)หารด้วยเวลาต่อเฟรม ได้ความเร็วลูกเป็น เมตร/วินาที และคำนวณระยะห่างแบบยูคลิด (Euclidean distance) ระหว่างผู้เล่นแต่ละคนกับลูกในทุกเฟรม

3.6 การแสดงผล

- หน้าต่างที่ 1 (2D Mapping Window) - แอนิเมชันแผนที่สนามจากมุมสูง แสดงตำแหน่งลูกและผู้เล่นทั้งสองพร้อมเส้นแนวการเคลื่อนที่ระยะสั้น (trail) ควบคู่กับวิดีโอต้นฉบับที่เล่นพร้อมกัน
- หน้าต่างที่ 2 (Information Window) - แผนที่ตำแหน่งผู้เล่น 1 และ 2 แยกกัน (สื่บงบอระยะห่างจากลูกในแต่ละจุด) และกราฟเส้นความเร็วลูกตามเวลา พร้อมเส้นประสีแดงกำกับเฟรมที่คาดว่าจะป็นจังหวะตีลูก

4. สรุปปัญหาที่พบ: การตรวจจับลูกเทนนิสผิดพลาด

ปัญหาหลักคือระบบตรวจจับลูกเทนนิสมักไปจับภาพ "ไฟสปอร์ตไลท์สนาม" (stadium light) แทนลูกเทนนิสจริง โดยบางครั้งให้ค่าความมั่นใจ (confidence) สูงกว่าลูกจริงเสียอีก และเกิดขึ้นแทบทุกเฟรม นอกจากนี้อัตราการตรวจจับลูกจริง (recall) ก็ต่ำมาก ตรวจไม่พบในเฟรมส่วนใหญ่ของวิดีโอ

สาเหตุที่แท้จริง (สรุปหลังสืบสวน)

สาเหตุที่แท้จริงไม่ใช่เพราะโมเดล "ไม่รู้จักร" ลูกเทนนิส แต่เป็นเรื่องความละเอียดตอนตรวจจับ (inference resolution): โค้ดเดิมเรียกใช้โมเดลตรวจจับทั้งเฟรม (1920x1080 พิกเซล) โดยไม่ระบุ imsz จึงใช้ค่าปริยายของ ultralytics คือ 640 พิกเซล การย่อภาพลงเกือบ 3 เท่าทำให้ทั้งลูกเทนนิสจริง (กล่องขนาดจริง ~25x29 พิกเซล) และไฟสนามที่ใหญ่กว่า มาก ถูกบีบจนเหลือเป็นก้อนกลมเบลอนขนาดใกล้เคียงกัน โมเดลจึงแยกไม่ออกและบางครั้งเลือกไฟสนามด้วยความมั่นใจที่สูงกว่า

ผลกระทบที่วัดได้จริง (ก่อนแก้ไข)

จากการทดสอบก่อนแก้ไข พบว่า 102 จาก 156 เฟรมที่ระบบรายงานว่า "พบลูก" (คิดเป็น 65%) แท้จริงแล้วเป็นตำแหน่งไฟสนามตำแหน่งเดียวกันซ้ำ ๆ ไม่ใช่ลูกเทนนิสจริงแต่อย่างใด และอัตราการตรวจจับลูกจริงมีเพียง 31 จาก 689 เฟรม (4.5%) เท่านั้น

5. ขั้นตอนการแก้ไขที่ทำไปแล้ว (เรียงตามลำดับเวลา)

5.1 ตัวกรองวัตถุนิ่ง (Static-Fixture Filter)

สถานะ: ใช้งานจริงอยู่ในไพพ์ไลน์ปัจจุบัน เป็นเกราะป้องกันชั้นที่สองควบคู่กับหัวข้อ 5.5

แนวคิด: ไฟสนามเป็นวัตถุที่อยู่นิ่งตำแหน่งฟิกเซลเดิมแทบทุกเฟรม ต่างจากลูกเทนนิสจริงที่เคลื่อนที่ผ่านแต่ละตำแหน่งเพียงชั่วคราว จึงสร้างระบบติดตามว่าตำแหน่งใด (แบ่งเป็นตารางกริดขนาด 12x12 พิกเซล) ถูกตรวจพบซ้ำบ่อยครั้งเพียงใด โดยมีค่าสลาย (decay) แบบทวีคูณทุกเฟรม หากตำแหน่งใดถูกตรวจพบสะสมเกินเกณฑ์จะถูกขึ้นบัญชีดำ (blacklist) และถูกปฏิเสธไม่ให้เลือกเป็นตำแหน่งลูกอีกต่อไป ไม่ว่าค่าความมั่นใจจากโมเดลจะสูงเพียงใดก็ตาม

OpenVCPipeline.py - ฟังก์ชัน `_register_ball_candidate`, `_is_static_fixture`

5.2 ความพยายาม: เทรนโมเดลลูกเทนนิสใหม่ด้วยตัวอย่างเชิงลบ (Hard-Negative Retraining)

สถานะ: เทรนเสร็จสมบูรณ์แล้ว แต่ผลการทดสอบจริงพบว่าไม่สามารถแก้ปัญหาหลักได้ จึงไม่ได้นำไปใช้งานจริง

สรุปขั้นตอน

- เก็บตัวอย่างภาพไฟสนาม (hard negatives) 23 ภาพจากตำแหน่งไฟที่ยืนยันแล้ว ขยายด้วยการแปรผัน (พลิกภาพ ปรับความสว่าง หมุนภาพ) เป็น 253 ภาพ
- พบและแก้ไขข้อผิดพลาด: พารามิเตอร์ fraction ของ ultralytics ตัดภาพชุดแรกตามลำดับตัวอักษรของชื่อไฟล์ ไม่ใช้การสุ่มจริง ต้องเปลี่ยนชื่อไฟล์ตัวอย่างเชิงลบให้ขึ้นต้นด้วย "00000000_" เพื่อบังคับให้ถูกเลือกใช้
- เทรน 30 epoch ที่ fraction=0.15 ผลลัพธ์บนชุด validation ดีขึ้นกว่ารอบแรกทุกตัวชี้วัด (precision 0.835 เป็น 0.915, recall 0.607 เป็น 0.749, mAP50 0.693 เป็น 0.802)

ข้อค้นพบ: โมเดลที่เทรนใหม่ยังคงล็อกเป้าไฟสนามอยู่ 100% ของเฟรม

ทดสอบโมเดลใหม่กับ video1.mp4 ทุกเฟรมโดยตรง พบว่ายังตรวจพบไฟสนามตำแหน่งเดิมใน 689 จาก 689 เฟรม (100%) ค่า validation ที่ดีขึ้นไม่ได้สะท้อนปัญหานี้เลย เพราะชุด validation ไม่มีภาพไฟสนามอยู่เลย สาเหตุที่คาดว่าเป็นไปได้: ตัวอย่างเชิงลบเป็นภาพครอบแยกขนาด 96x96 พิกเซล ซึ่งสอนโมเดลในบริบทที่ต่างจากการใช้งานจริงมากเกินไป

5.3 การแก้ไขที่ได้ผล ขั้นที่ 1: เพิ่มความละเอียดการตรวจจับ (imgsz=1280)

พบว่า การเรียกโมเดลตรวจจับทั้งเฟรมโดยไม่ระบุ imgsz ทำให้ใช้ค่าปริยาย 640 พิกเซล ย่อภาพจาก 1920 พิกเซลลงเกือบ 3 เท่า จนลูกเทนนิสจริงและไฟสนามดูใกล้เคียงกันเกินไป การเปลี่ยนไปใช้ imgsz=1280 (คงความละเอียดไว้มากกว่าเดิม) ทำให้แยกออกจากกันได้ชัดเจนขึ้นมาก โดยไม่ต้องเทรนโมเดลใหม่เลย

ตัวชี้วัด	ก่อนแก้ (imgsz ปริยาย 640)	หลังแก้ (imgsz=1280)
เฟรมที่พบลูกจริง	31 / 689 (4.5%)	174 / 689 (25%)
ไฟสนามดวงที่ 1 (656,299)	65% ของที่พบ	0%
ไฟสนามดวงที่ 2 (1746,330)	-	~12% ของกล้องดิบ (ยังไม่ถูกเลือก)

ข้อสังเกต: ยังพบจุดผิดพลาดใหม่แบบเล็กน้อยคือเครื่องหมายกึ่งกลางเส้นหลัง (baseline center mark) ถูกเข้าใจผิดว่าเป็นลูกใน 9% ของเฟรมที่ตรวจพบ และไฟสนามดวงที่สองยังคงถูกตรวจพบเป็นกล่องติดอยู่บ้าง (แม้ตัวกรองวัตถุหนึ่งจะกันไม่ให้ถูกเลือกเป็นคำตอบสุดท้าย) จึงยังไม่ใช่วางแก้ที่สมบูรณ์ที่สุด

5.4 แนวคิดที่นำไปสู่ทางแก้สุดท้าย

ผู้ใช้เสนอแนวคิด: "ครอบไฟสนามออก แล้วซูมเข้าไปในแต่ละเฟรมเพื่อหาลูก เหมือนกับที่ทำกับผู้เล่นฝั่งไกล"

เมื่อวิเคราะห์แนวคิดนี้ พบว่าหลักการเบื้องต้นถูกต้อง (ให้โมเดลเห็นพิกเซลของวัตถุมากขึ้น = ตรวจจับได้แม่นยำขึ้น)

ซึ่งตรงกับเหตุผลที่ `imgsz=1280` ช่วยได้มากในหัวข้อ 5.3 อยู่แล้ว แต่การ "ครอบจุดเดียว" แบบผู้เล่นฝั่งไกลใช้ไม่ได้ตรง ๆ กับลูกเทนนิส เพราะผู้เล่นฝั่งไกลอยู่ในบริเวณที่แน่นอนตายตัว (ครึ่งสนามฝั่งไกล) ในขณะที่ลูกเทนนิสเคลื่อนที่ได้ทั่วทั้งสนาม ไม่มีจุดครอบเดียวที่ครอบคลุมตำแหน่งลูกได้เสมอ

บทสรุปคือ นำแนวคิดไปประยุกต์ให้ถูกต้อง: แทนที่จะครอบจุดเดียว ให้แบ่งบริเวณที่เป็นไปได้ของลูกทั้งหมดออกเป็นชิ้นย่อย (tiles) หลายจุดซ้อนทับกันเล็กน้อย แล้วตรวจจับทีละชิ้นที่ความละเอียดต้นฉบับ นี่คือเทคนิคที่เรียกว่า tiled/sliced inference ซึ่งเป็นวิธีมาตรฐานสำหรับปัญหาการตรวจจับวัตถุขนาดเล็กในภาพขนาดใหญ่

5.5 การแก้ไขที่ได้ผลจริง ขั้นสุดท้าย: ตรวจจับแบบแบ่งส่วนภาพ (Tiled Detection)

ผลลัพธ์ที่ดีที่สุดเท่าที่ทำได้ - ไม่มีจุดผิดพลาดที่รู้จักเหลืออยู่เลย

แบ่งบริเวณที่เป็นไปได้ของลูก (plausible region) ออกเป็นชิ้นย่อยขนาด 640×640 พิกเซล ซ้อนทับกัน 100 พิกเซล (ในวิดีโอนี้ได้ทั้งหมด 8 ชิ้น) ตรวจจับทีละชิ้นที่ความละเอียดต้นฉบับ (ไม่ย่อภาพเลย) แล้วรวมผลลัพธ์กลับเป็นพิกัดของภาพเต็ม ก่อนส่งต่อให้ตัวกรองเดิม (plausible-region + static-fixture) ทำงานตามปกติ

OpenVCPipeline.py - ฟังก์ชัน `detect_ball_tiled`

ผลการทดสอบเปรียบเทียบทั้ง 3 ชั้น (รันไพล์ไลน์เต็มรูปแบบกับ video1.mp4 จริงทุกครั้ง)

ตัวชี้วัด	เดิม	<code>imgsz=1280</code>	แบ่งส่วนภาพ (สุดท้าย)
เฟรมที่พบลูกจริง	31 (4.5%)	174 (25%)	307 (44.6%)
ไฟสนามดวงที่ 1	65%	0%	0%
ไฟสนามดวงที่ 2	-	~12% ดิบ	0%
เครื่องหมายกึ่งกลางเส้น	-	9%	0%

ตรวจสอบคลัสเตอร์ตำแหน่งที่พบบ่อยที่สุดในผลลัพธ์ล่าสุดด้วยมือ พบว่าเป็นลูกเทนนิสจริง (ตำแหน่งเคลื่อนที่ต่อเนื่อง

ราบรื่นภายในแต่ละช่วงที่ตรวจพบ เพียงแต่ผ่านบริเวณใกล้เคียงกันของสนามหลายครั้งในการได้ลูกหลายครั้ง)

ไม่ใช่จุดผิดพลาดใหม่ ข้อแลกเปลี่ยน: การตรวจจับลูกช้าลงกว่า `imgsz=1280` ประมาณ 2.5 เท่า (รวมทั้งไพล์ไลน์ใช้เวลาประมาณ 7 นาทีสำหรับวิดีโอความยาว 11.5 วินาทีบน CPU เครื่องนี้) ซึ่งยอมรับได้เนื่องจากเป็นการประมวลผลแบบออฟไลน์

6. สถานะปัจจุบันและขั้นตอนถัดไป

สรุป: ปัญหาการตรวจจับลูกเทนนิสได้รับการแก้ไขอย่างสมบูรณ์สำหรับวิดีโอ

ไพอ์ปไลน์ที่ใช้งานจริงตอนนี้ตรวจจับลูกด้วยวิธีแบ่งส่วนภาพ (detect_ball_tiled หัวข้อ 5.5) ร่วมกับตัวกรองวัตถุเป็นเกราะป้องกันชั้นที่สอง (หัวข้อ 5.1) ผลทดสอบเต็มรูปแบบกับ video1.mp4 ล่าสุดยืนยันว่าไม่มีจุดผิดพลาดที่เคยพบ (ไฟสนามทั้งสองดวง และเครื่องหมายกึ่งกลางเส้นหลัง) เหลืออยู่เลย และอัตราการตรวจพบลูกจริงเพิ่มขึ้นเป็น 44.6% จากเดิมเพียง 4.5% ส่วนโมเดลที่เทรนขึ้นใหม่เฉพาะทาง (หัวข้อ 5.2) ไม่ได้ถูกนำไปใช้งาน เนื่องจากทดสอบแล้วไม่ได้ผล

- ไฟล์ผลลัพธ์ล่าสุดจากการรันด้วยการแก้ไขนี้: detected_video_10.mp4, court_coordinates_10.csv, player_tracking_output_8.csv
- Mapping.py ได้ปรับให้ชี้ไปยังไฟล์ผลลัพธ์ชุดล่าสุดแล้ว
- อัตราการตรวจพบ 44.6% ยังไม่ใช่ 100% - ยังมีบางเฟรมที่ลูกเคลื่อนที่เร็วเกินไปหรือมีขนาดเล็กเกินกว่าจะตรวจจับได้ หากต้องการปรับปรุงต่อ แนวทางที่เป็นไปได้คือการเติมช่องว่างด้วยการทำนายเส้นทางการเคลื่อนที่ (trajectory-based interpolation) จากเฟรมที่ตรวจพบแล้วรอบข้าง แทนที่จะพยายามให้ตรวจจับได้ทุกเฟรม

ไฟล์ที่เกี่ยวข้อง

- OpenVCPipeline.py - ไพอ์ปไลน์หลัก: ตรวจจับเส้นสนาม ผู้เล่น และลูกเทนนิสด้วยวิธีแบ่งส่วนภาพ (ใช้งานจริงอยู่)
- Mapping.py - แปลงพิกัดเป็นเมตรและแสดงผลแผนที่สนาม 2 มิติ (ชี้ไปยังผลลัพธ์ชุดล่าสุดแล้ว)
- mine_hard_negatives.py - สคริปต์เก็บตัวอย่างภาพไฟสนามจากวิดีโอ (ใช้ในความพยายามหัวข้อ 5.2 เท่านั้น)
- train_ball_model.py - สคริปต์เทรนโมเดลตรวจจับลูกเทนนิสของตัวเอง (ผลลัพธ์ไม่ได้นำไปใช้งานจริง)
- ball_training_runs/tennis_ball_yolov8n_v2 - ผลลัพธ์การเทรนรอบใหม่ (เทรนเสร็จแล้ว แต่ไม่ได้ใช้งานจริง)